Laboratory Work 1

International Airport Database Entity Relationship Diagram

The database design manages airports, airlines, flights, passenger bookings, boarding passes, baggage, and security checks. The diagram below was prepared in draw.io and shows ten entities and twelve relationships.

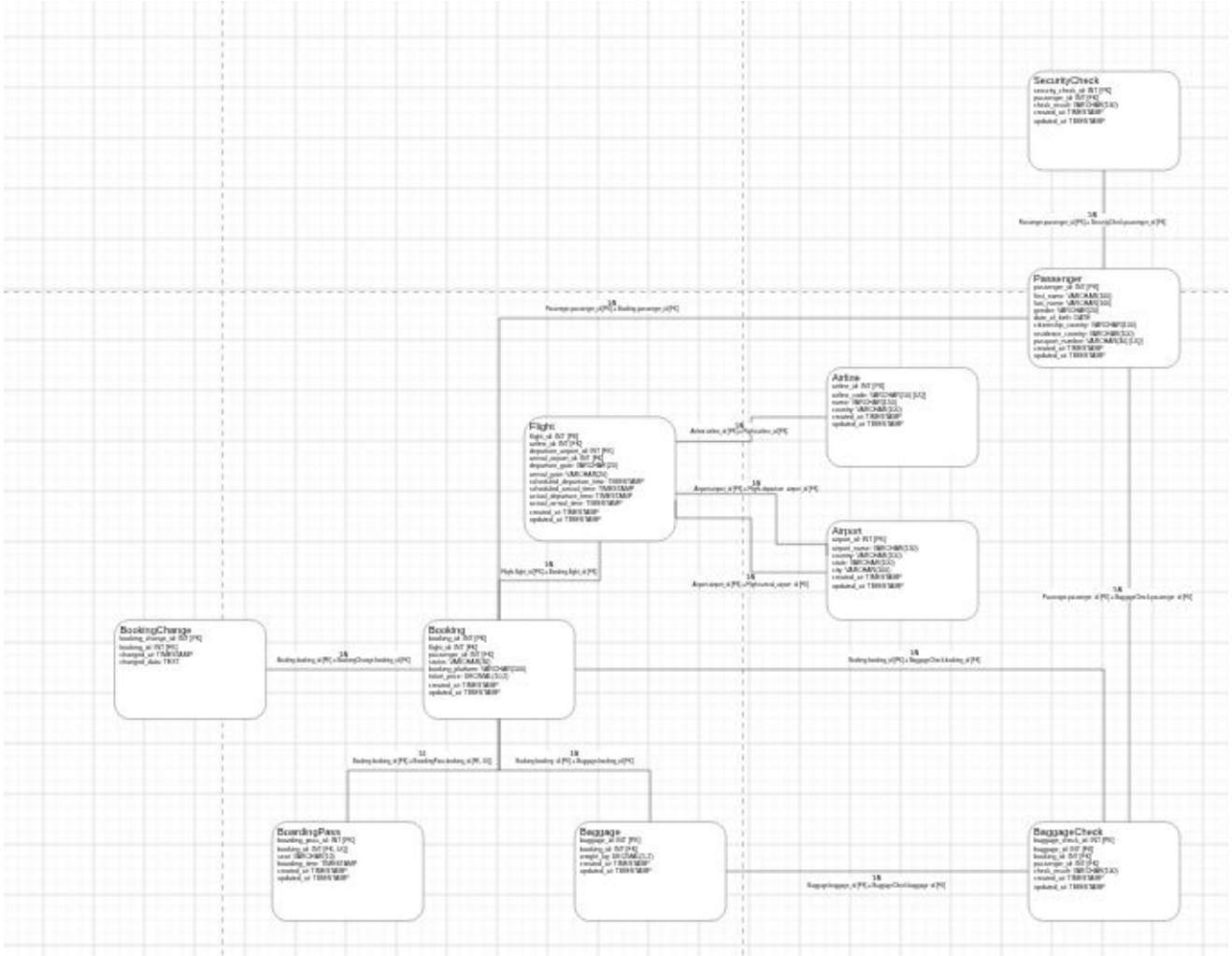


Figure 1. ER diagram exported from the submitted draw.io source. PK - primary key; FK - foreign key; UQ - unique constraint.

Entities and Attributes

Each entity is represented as a table. Primary keys uniquely identify records. Foreign keys reference primary keys in related tables, and unique constraints prevent duplicate values where required.

Entity	Purpose	Attributes and constraints
Airport	Stores airport identity, name, geographical location, and record timestamps.	airport_id: INT [PK] airport_name: VARCHAR(150) country: VARCHAR(100) state: VARCHAR(100) city: VARCHAR(100) created_at: TIMESTAMP updated_at: TIMESTAMP
Airline	Stores airline identity, unique airline code, name, country, and record timestamps.	airline_id: INT [PK] airline_code: VARCHAR(10) [UQ] name: VARCHAR(150) country: VARCHAR(100) created_at: TIMESTAMP updated_at: TIMESTAMP
Flight	Stores a scheduled flight, its airline, departure and arrival airports, gates, planned times, and actual times.	flight_id: INT [PK] airline_id: INT [FK] departure_airport_id: INT [FK] arrival_airport_id: INT [FK] departure_gate: VARCHAR(20) arrival_gate: VARCHAR(20) scheduled_departure_time: TIMESTAMP scheduled_arrival_time: TIMESTAMP actual_departure_time: TIMESTAMP actual_arrival_time: TIMESTAMP created_at: TIMESTAMP updated_at: TIMESTAMP
Passenger	Stores passenger identity, personal details, citizenship, residence, and a unique passport number.	passenger_id: INT [PK] first_name: VARCHAR(100) last_name: VARCHAR(100) gender: VARCHAR(20) date_of_birth: DATE citizenship_country: VARCHAR(100) residence_country: VARCHAR(100) passport_number: VARCHAR(30) [UQ] created_at: TIMESTAMP updated_at: TIMESTAMP

Entity	Purpose	Attributes and constraints
Booking	Connects one passenger with one flight and stores booking status, platform, ticket price, and timestamps.	booking_id: INT [PK] flight_id: INT [FK] passenger_id: INT [FK] status: VARCHAR(30) booking_platform: VARCHAR(100) ticket_price: DECIMAL(10,2) created_at: TIMESTAMP updated_at: TIMESTAMP
BookingChange	Keeps a history of changes made to a booking.	booking_change_id: INT [PK] booking_id: INT [FK] changed_at: TIMESTAMP changed_data: TEXT
BoardingPass	Stores the seat and boarding time for the boarding pass issued for a booking.	boarding_pass_id: INT [PK] booking_id: INT [FK, UQ] seat: VARCHAR(10) boarding_time: TIMESTAMP created_at: TIMESTAMP updated_at: TIMESTAMP
Baggage	Stores each baggage item registered under a booking, including its weight.	baggage_id: INT [PK] booking_id: INT [FK] weight_kg: DECIMAL(5,2) created_at: TIMESTAMP updated_at: TIMESTAMP
BaggageCheck	Stores baggage inspection results and connects the check to baggage, booking, and passenger records.	baggage_check_id: INT [PK] baggage_id: INT [FK] booking_id: INT [FK] passenger_id: INT [FK] check_result: VARCHAR(100) created_at: TIMESTAMP updated_at: TIMESTAMP
SecurityCheck	Stores the result of a passenger security check.	security_check_id: INT [PK] passenger_id: INT [FK] check_result: VARCHAR(100) created_at: TIMESTAMP updated_at: TIMESTAMP

Relationships and Cardinality

A 1:N relationship means that one record in the parent table can be referenced by many records in the child table. A 1:1 relationship means that a record can be associated with only one record on the other side. Every relationship below links a primary key to the corresponding foreign key.

Cardinality	Connected fields	Meaning
1:N	Airline.airline_id [PK] = Flight.airline_id [FK]	One airline can operate many flights; each flight belongs to one airline.
1:N	Airport.airport_id [PK] = Flight.departure_airport_id [FK]	One airport can be the departure airport for many flights.
1:N	Airport.airport_id [PK] = Flight.arrival_airport_id [FK]	One airport can be the arrival airport for many flights.
1:N	Flight.flight_id [PK] = Booking.flight_id [FK]	One flight can have many bookings; each booking refers to one flight.
1:N	Passenger.passenger_id [PK] = Booking.passenger_id [FK]	One passenger can make many bookings; each booking belongs to one passenger.
1:N	Booking.booking_id [PK] = BookingChange.booking_id [FK]	One booking can have many entries in its change history.
1:1	Booking.booking_id [PK] = BoardingPass.booking_id [FK, UQ]	A unique booking reference makes the Booking-BoardingPass relationship one-to-one.
1:N	Booking.booking_id [PK] = Baggage.booking_id [FK]	One booking can include multiple baggage items.
1:N	Baggage.baggage_id [PK] = BaggageCheck.baggage_id [FK]	One baggage item can have multiple inspection records.
1:N	Passenger.passenger_id [PK] = BaggageCheck.passenger_id [FK]	One passenger can be associated with multiple baggage checks.
1:N	Passenger.passenger_id [PK] = SecurityCheck.passenger_id [FK]	One passenger can have multiple security check records over time.
1:N	Booking.booking_id [PK] = BaggageCheck.booking_id [FK]	One booking can be associated with multiple baggage check records.

Design Decisions

Two airport roles. Flight contains departure_airport_id and arrival_airport_id. Both are foreign keys that reference Airport.airport_id, but they represent different roles: the airport of departure and the airport of arrival.

Booking history. BookingChange is separated from Booking so that multiple changes can be stored without overwriting the current booking record.

Boarding pass uniqueness. BoardingPass.booking_id is both a foreign key and unique, enforcing one boarding pass per booking in this simplified model.

Baggage check identification. baggage_id is included so that each inspection identifies the exact baggage item being checked. booking_id and passenger_id preserve the direct links required by the task description.